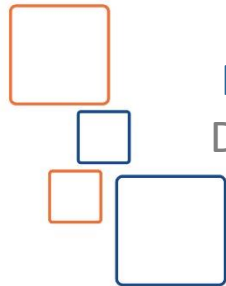


CORE JAVA PROGRAMMING

AN INTRODUCTION TO JAVA



Presented By
Dr. Eman Hesham, DBA.



Java™ Education
and Technology Services



Invest In Yourself,
Develop Your Career

Agenda

- A. The History and Evolution of Java
- B. An Overview of Java
- C. Data Types, Variables and Arrays
- D. Operators, Control Statements and String Handling
- E. Modifiers, Access Specifiers, Packages and Interfaces
- F. Wrapper Classes, Autoboxing and Inner Classes
- G. Exception Handling
- H. Generics and Lambda Expressions
- I. Java Stream API
- J. Java Collections
- K. Multi-Threading in Java

Agenda

- A. The History and Evolution of Java
- B. An Overview of Java
- C. Data Types, Variables and Arrays
- D. Operators, Control Statements and String Handling
- E. Modifiers, Access Specifiers, Packages and Interfaces
- F. Wrapper Classes, Autoboxing and Inner Classes
- G. Exception Handling
- H. Generics and Lambda Expressions and Method Reference
- I. Java Stream API
- J. Java Collections**
- K. Multi-Threading in Java

Lesson 10

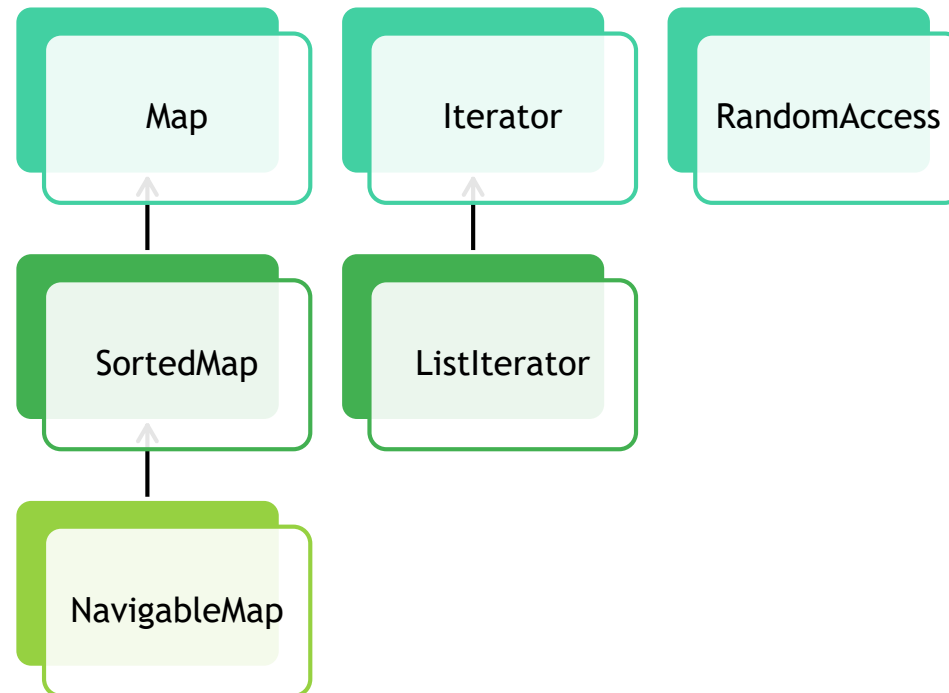
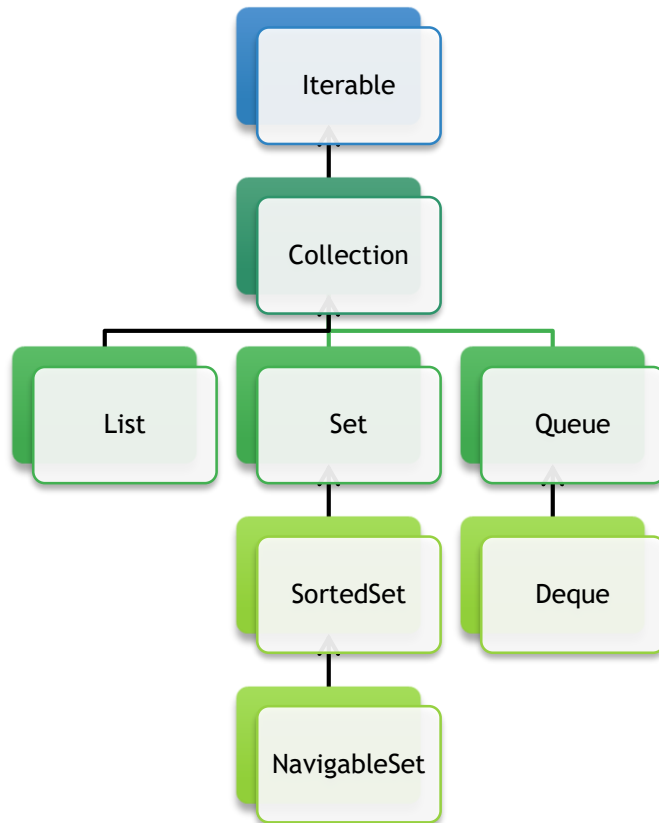
Java Collections



Introduction to Java Collection Framework

- ▶ The data structures that you choose can make a big difference when you try to implement methods in a natural style or are concerned with performance.
 - Do you need to search quickly through thousands (or even millions) of sorted items?
 - Do you need to rapidly insert and remove elements in the middle of an ordered sequence?
 - Do you need to establish associations between keys and values?

Interfaces in the Collections Framework



Interfaces in the Collections Framework

- ▶ A List is an *ordered collection*. Elements are added into a particular position in the container.
- ▶ The **ListIterator** interface is a subinterface of Iterator. It defines a method for adding an element before the iterator position.
- ▶ The Set interface is identical to the Collection interface, but the behavior of the methods is more tightly defined. *The add method of a set should reject duplicates.*

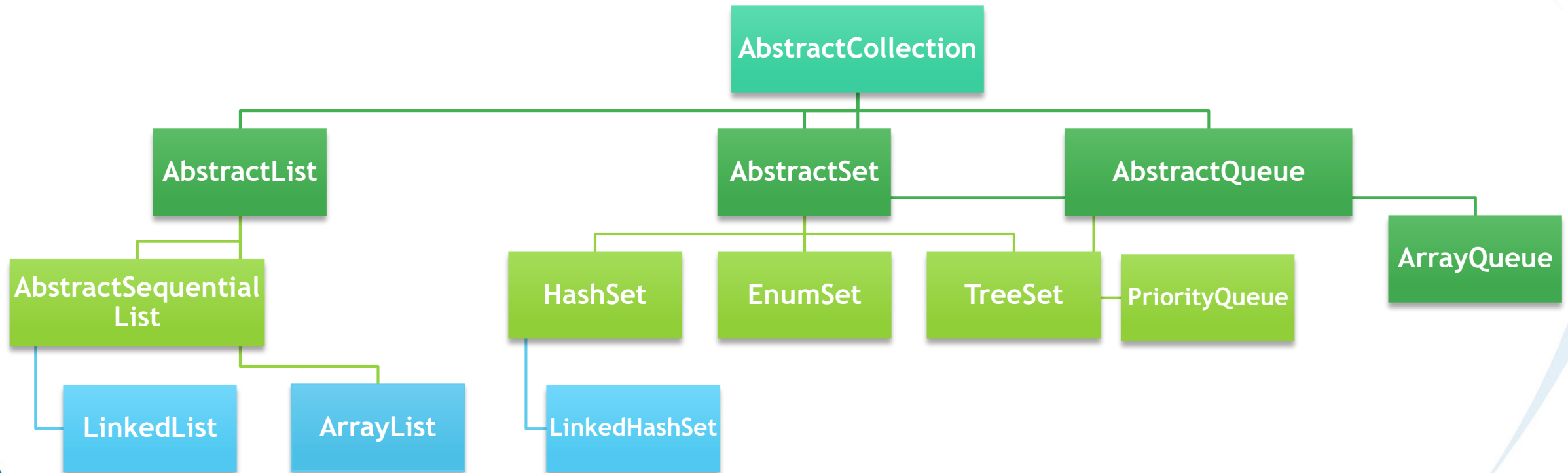
Interfaces in the Collections Framework

- ▶ The ***SortedSet*** and ***SortedMap*** interfaces expose the comparator object used for sorting, and they define methods to obtain views of subsets of the collections.
- ▶ Java SE 6 introduced interfaces ***NavigableSet*** and ***NavigableMap*** that contain additional methods for searching and traversal in sorted sets and maps.
- ▶ The ***TreeSet*** and ***TreeMap*** classes implement these interfaces.

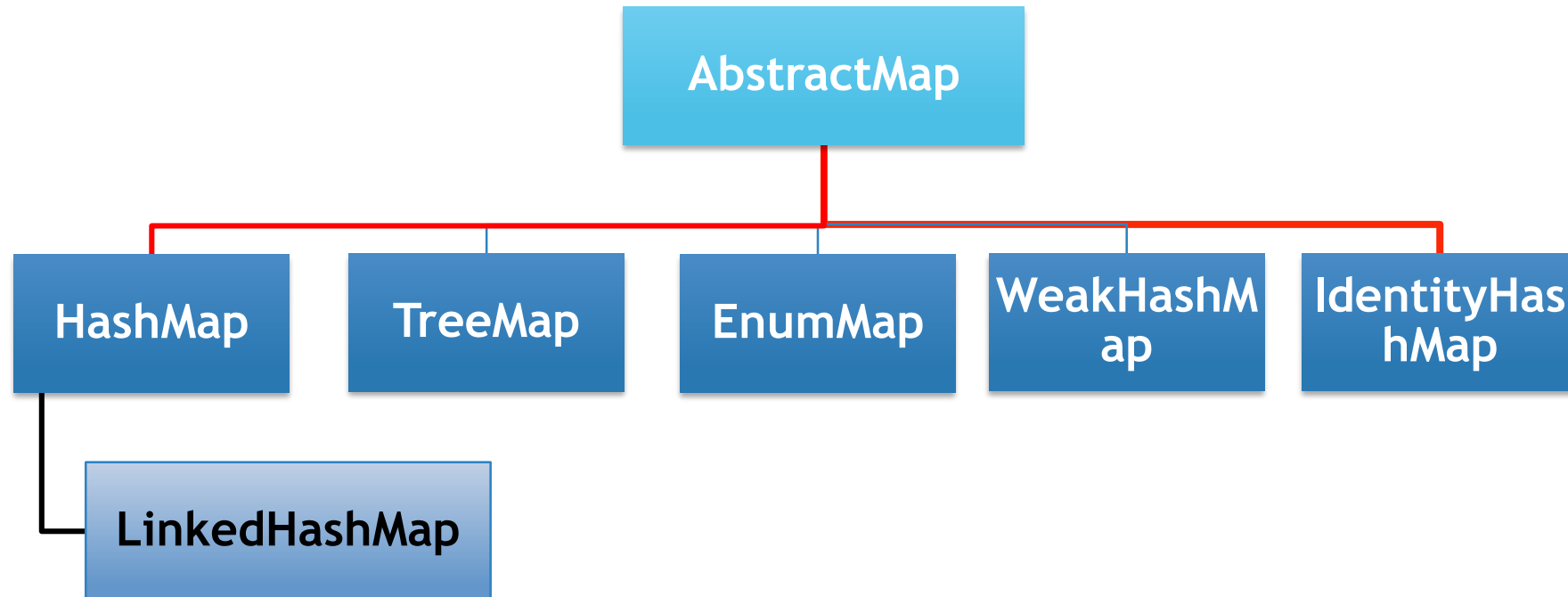
Concrete Collections

- ▶ The following slides show the collections in the Java library and briefly describes the purpose of each collection class.
- ▶ All classes in the below table implement the *Collection* interface, with the exception of the classes with names ending in *Map*. Those classes implement the Map interface instead.

Classes in the collections framework



Classes in the collections framework



Classes in the collections framework

Collection Type	Description
ArrayList	An indexed sequence that grows and shrinks dynamically
LinkedList	An ordered sequence that allows efficient insertion and removal at any location
ArrayDeque	A double-ended queue that is implemented as a circular array
HashSet	An unordered collection that rejects duplicates
TreeSet	A sorted set
EnumSet	A set of enumerated type values
LinkedHashSet	A set that remembers the order in which elements were inserted
PriorityQueue	Elements are ordered according to their natural ordering, or by a Comparator provided at queue construction time

Classes in the collections framework

Collection Type	Description
HashMap	A data structure that stores key/value associations
TreeMap	A map in which the keys are sorted
EnumMap	A map in which the keys belong to an enumerated type
LinkedHashMap	A map that remembers the order in which entries were added
WeakHashMap	A map with values that can be reclaimed by the garbage collector if they are not used elsewhere
IdentityHashMap	A map with keys that are compared by ==, not equals

Linked Lists

- ▶ Arrays and ArrayList are used frequently when we are developing different applications.
- ▶ However, arrays and array lists suffer from a major drawback.
- ▶ Removing an element from the middle of an array is expensive since all array elements beyond the removed one must be moved toward the beginning of the array. The same is true for inserting elements in the middle.
- ▶ Another well-known data structure, the *linked list*, solves this problem.

Linked Lists

- ▶ While an array stores object references in consecutive memory locations, a linked list stores each object in a separate link.
- ▶ Each link also stores a reference to the next link in the sequence.
- ▶ In the Java programming language, all linked lists are actually doubly linked; that is, each link also stores a reference to its predecessor
- ▶ Removing an element from the middle of a linked list is an inexpensive operation—only the links around the element to be removed need to be updated



Doubly Linked List

Linked Lists

- ▶ The following code example adds three elements and then removes the second one:

```
// LinkedList implements List
List<String> staff = new LinkedList<>();
staff.add("First");
staff.add(new String("Second"));
staff.add("Third");
Iterator iter = staff.iterator();
String first = iter.next(); // visit first element
String second = iter.next(); // visit second element
iter.remove(); // remove last visited element
```


Linked Lists

- ▶ A linked list is an ordered collection in which the position of the objects matters.
- ▶ The `LinkedList.add` method adds the object to the end of the list. But you will often want to add objects somewhere in the middle of a list.
- ▶ This position dependent add method is the responsibility of an iterator, since iterators describe positions in collections.
- ▶ Using iterators to add elements makes sense only for collections that have a natural ordering.

Linked Lists

```
List<String> numbers = new LinkedList<>();  
numbers.add("First");  
numbers.add("Second");  
numbers.add("Third");  
ListIterator<String> iter = numbers.listIterator();  
iter.next(); // skip past first element  
iter.add("Before Second");
```

Linked Lists - Example

- ▶ The following program puts linked lists to work.
- ▶ It simply creates two lists, merges them, then removes every second element from the second list, and finally tests the removeAll method.

Linked Lists - Example

```
import java.util.*;

public class LinkedListTest
{
    public static void main(String[] args)
    {
        List<String> a = new LinkedList<>();
        a.add("A");
        a.add("B");
        a.add("C");
        List<String> b = new LinkedList<>();
        b.add("D");
        b.add("E");
        b.add("F");
        b.add("G");
    }
}
```

Linked Lists - Example

```
// merge the words from b into a
ListIterator<String> aIter = a.listIterator();
Iterator<String> bIter = b.iterator();
while (bIter.hasNext()){
    if (aIter.hasNext()) aIter.next();
    aIter.add(bIter.next());
}
System.out.println(a);
// remove every second word from b
bIter = b.iterator();
while (bIter.hasNext()){
    bIter.next(); // skip one element
    if (bIter.hasNext())
    {
        bIter.next(); // skip next element
        bIter.remove(); // remove that element
    }
}
System.out.println(b);
```

[A, D, B, E, C, F, G]

[D, F]

Linked Lists - Example

```
// bulk operation: remove all words in b from a
a.removeAll(b);
System.out.println(a);
}
}
```

[A, B, E, C, G]

Lists

- ▶ **Hint:** Java List is an ordered collection.
- ▶ Provides control over the position where you can insert an element.
- ▶ List allows you to add **duplicate** elements.
- ▶ List allows you to have '**null**' elements.
- ▶ List interface got many default methods, for example **replaceAll**, **sort** and **splititerator**.
- ▶ List indexes start from **0**, just like arrays.
- ▶ List supports **Generics** and we should use it whenever possible.
- ▶ Using Generics with List will avoid **ClassCastException** at runtime.

Maps

- ▶ A set is a collection that lets you quickly find an existing element.
- ▶ However, to look up an element, you need to have an exact copy of the element to find.
- ▶ That isn't a very common lookup—usually, you have some key information, and you want to look up the associated element.
- ▶ The map data structure serves that purpose.
- ▶ A map stores **key/value** pairs. You can find a value if you provide the key.

Basic Map Operations

- ▶ The Java library supplies two general-purpose implementations for maps: **HashMap** and **TreeMap**. Both classes implement the Map interface.
- ▶ A hash map hashes the keys, and a tree map uses an ordering on the keys to organize them in a search tree.
- ▶ The hash or comparison function is applied only to the keys. The values associated with the keys are not hashed or compared.

Basic Map Operations

- ▶ Here is how you set up a hash map for storing employees:

```
//HashMap implements Map  
Map<String, Employee> staff = new HashMap<>();  
Employee amr = new Employee("Amr Mostafa");  
staff.put("987-98-9996", amr);
```

- ▶ Whenever you add an object to a map, you must supply a key as well. In our case, the key is a string, and the corresponding value is an Employee object.

Basic Map Operations

- ▶ To retrieve an object, you must use (and, therefore, remember) the key.

```
String id = "987-98-9996";  
e = staff.get(id);
```

If no information is stored in the map with the particular key specified, get returns null.

Basic Map Operations

- ▶ The null return value can be inconvenient.
- ▶ Sometimes, you have a good default that can be used for keys that are not present in the map. Then use the `getOrDefault` method.

```
Map<String, Integer> scores = . . . ;  
int score = scores.get(id, 0); // Gets 0 if the id is not present
```
- ▶ **Keys** must be **unique**. You cannot store two values with the same key.
- ▶ If you call the `put` method twice with the same key, the second value **replaces** the first one.
- ▶ In fact, `put` returns the previous value associated with its key parameter.

Basic Map Operations

- ▶ The remove method removes an element with a given key from the map.
- ▶ The size method returns the number of entries in the map.
- ▶ The easiest way of iterating over the keys and values of a map is the forEach method.
 - ▶ Provide a lambda expression that receives a key and a value. That expression is invoked for each map entry in turn.

```
scores.forEach((k, v) -> System.out.println("key=" + k + ", value=" + v));
```

Basic Map Operations - Example

- ▶ We first add key/value pairs to a map.
- ▶ Then, we remove one key from the map, which removes its associated value as well.
- ▶ Next, we change the value that is associated with a key and call the get method to look up a value. Finally, we iterate through the entry set.

Basic Map Operations - Example

```
public class Employee {  
    private String name;  
    public Employee(String name) {  
  
        this.setName(name);  
    }  
    public String getName() {  
        return name;  
    }  
    public void setName(String name) {  
        this.name = name;  
    }  
    public String toString() {  
        return "Employee Name "+name;  
    }  
}
```

Basic Map Operations - Example

```
import java.util.*;
public class MapTest {
    public static void main(String[] args) {
        Map<String, Employee> staff = new HashMap<>();
        staff.put("144-25-5464", new Employee("Amr"));
        staff.put("567-24-2546", new Employee("Mohsen"));
        staff.put("157-62-7935", new Employee("Medhat"));
        staff.put("456-62-5527", new Employee("Eman"));
    }
}
```


Basic Map Operations - Example

```
// print all entries
System.out.println(staff);
// remove an entry
staff.remove("567-24-2546");
// replace an entry
staff.put("456-62-5527", new Employee("Mona"));
// look up a value
System.out.println(staff.get("157-62-7935"));
// iterate through all entries
staff.forEach((k, v) ->
System.out.println("key=" + k + ", value=" + v));
}
```

Map Example-Output

```
{157-62-7935=collections.maps.Employee@7960847b, 144-25-5464=collections.maps.Employee@6a6824be, 456-62-5527=collections.maps.Employee@5c8da962, 567-24-2546=collections.maps.Employee@512ddf17}
collections.maps.Employee@7960847b
key=157-62-7935,
value=collections.maps.Employee@7960847b
key=144-25-5464,
```

```
{157-62-7935=Medhat, 144-25-5464=Amr, 456-62-5527=Iman, 567-24-2546=Mohsen}
Medhat
key=157-62-7935, value=Medhat
key=144-25-5464, value=Amr
key=456-62-5527, value=Mona
```

Lesson 9

Java Stream API



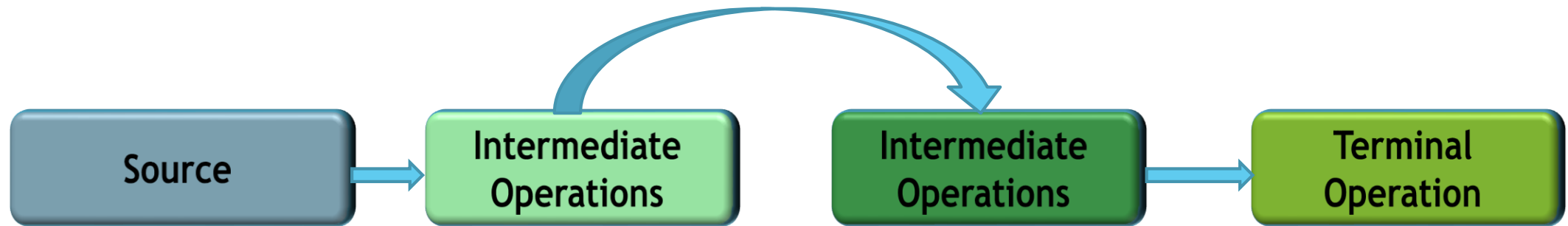
Stream API

- ▶ Java SE 8 introduces the concept of streams.
- ▶ Streams are objects of classes that implement interface Stream (from the package *java.util.stream*) or one of the specialized stream interfaces for processing collections of int, long or double values.
- ▶ Together with lambdas, streams enable you to perform tasks on collections of elements—often from an array or collection object.

Stream API: Stream Pipelines

- ▶ Streams move elements through a sequence of processing steps—known as a stream pipeline—that begins with a data source (such as an array or collection), performs various intermediate operations on the data source's elements and ends with a terminal operation.
- ▶ A stream pipeline is formed by chaining method calls.
- ▶ Unlike collections, ***streams do not have their own storage***—once a stream is processed, it cannot be reused, because it does not maintain a copy of the original data source.

Stream API: Stream Pipelines



```
.stream()  
  .filter(b -> b.getColor() == red)  
  .mapToInt( b -> b.getWeight() )  
  .sum();
```

Stream API: Intermediate Operations

- ▶ An intermediate operation specifies tasks to perform on the stream's elements and always results in a **new stream**.
- ▶ Intermediate operations are lazy—they aren't performed until a terminal operation is invoked.
- ▶ This allows library developers to optimize stream-processing performance.

Stream API: Intermediate Operations

- ▶ For example, if you have a collection of 1,000,000 Person objects and you're looking for the first one with the last name "Jones", stream processing can terminate as soon as the first such Person object is found.

Stream API: Terminal Operations

- ▶ A terminal operation initiates processing of a stream pipeline's intermediate operations and produces a result.
- ▶ Terminal operations are eager—they perform the requested operation when they are called.
- ▶ The Below tables show some common intermediate and terminal operations

Stream API: Common Intermediate Stream Operations

Operation	Description
Filter	Results in a stream containing only the elements that satisfy a condition.
distinct	Results in a stream containing only the unique elements.
limit	Results in a stream with the specified number of elements from the beginning of the original stream.
map	Results in a stream in which each element of the original stream is mapped to a new value (possibly of a different type). The new stream has the same number of elements as the original stream.
sorted	Results in a stream in which the elements are in sorted order. The new stream has the same number of elements as the original stream.

Stream API: Common Terminal Stream Operations

- ▶ Loop
 - ▶ `forEach`- Performs processing on every element in a stream (e.g., display each element).
- ▶ Reduction operations
 - ▶ Take all values in the stream and return a single value
- ▶ Mutable reduction operations
 - ▶ Create a container (such as a collection or `StringBuilder`)
- ▶ Search operations
 - ▶ Performs different search or match operations on a stream

Stream API: Common Terminal Stream Operations

Operation	Reduction operations
average	Calculates the average of the elements in a numeric stream.
count	Returns the number of elements in the stream.
max	Locates the largest value in a numeric stream.
min	Locates the smallest value in a numeric stream.
reduce	Reduces the elements of a collection to a single value using an associative accumulation function (e.g., a lambda that adds two elements).

Stream API: Common Terminal Stream Operations

Operation	Mutable Reduction operations
collect	Creates a new collection of elements containing the results of the stream's prior operations.
toArray	Creates an array containing the results of the stream's prior operations..

Stream API: Common Terminal Stream Operations

Operation	Search operations
findFirst	Finds the first stream element based on the prior intermediate operations; immediately terminates processing of the stream pipeline once such an element is found.
findAny	Finds any stream element based on the prior intermediate operations; immediately terminates processing of the stream pipeline once such an element is found.
anyMatch	Determines whether any stream elements match a specified condition; immediately terminates processing of the stream pipeline if an element matches.
allMatch	Determines whether all of the elements in the stream match a specified condition.

Stream API Example

- ▶ In the following example we will show how to use Stream API and lambda expressions to simplify programming tasks that you probably already performed for *Arrays* and *ArrayLists*
- ▶ The Example demonstrates operations on an *IntStream* (package *java.util.stream*)—a specialized stream for manipulating int values.
- ▶ The techniques shown in this example also apply to *LongStreams* and *DoubleStreams* for long and double values, respectively.

Stream API Example

```
import java.util.stream.IntStream;

public class IntStreamOperations {

    public static void main(String[] args) {

        int[] values = {3, 10, 6, 1, 4, 8, 2, 5, 9, 7};

        // display original values

        System.out.print("Original values: ");

        IntStream.of(values).forEach(value -> System.out.printf("%d ", value));

        System.out.println();
    }
}
```

```
1001
Original values: 3 10 6 1 4 8 2 5 9 7
*****
```


Stream API Example

//count, min, max, sum and average of the values

```
System.out.printf("%nCount: %d%n", IntStream.of(values).count());
```

```
System.out.printf("Min: %d%n",  
IntStream.of(values).min().getAsInt());
```

```
System.out.printf("Max: %d%n",  
IntStream.of(values).max().getAsInt());
```

```
System.out.printf("Sum: %d%n", IntStream.of(values).sum());
```

```
System.out.printf("Average: %.2f%n",  
IntStream.of(values).average().getAsDouble());
```

Returns OptionalInt which is a container object which may or may not contain an int value

```
Count: 10  
Min: 1  
Max: 10  
Sum: 55  
Average: 5.50
```

Stream API Example

The identity value must be an identity for the accumulator function. This means that for all x , `accumulator.apply(identity, x)` is equal to x . The accumulator function must be an [associative function](#).

```
//sum of squares of values with reduce method
```

```
System.out.printf("Sum of squares via reduce method:
```

```
%d%n", IntStream.of(values)
```

```
.reduce(0, (x, y) -> x+y*y));
```

```
//product of values with reduce method
```

```
System.out.printf("Product via reduce method: %d%n", IntStream.of(values)
```

```
.reduce(1, (x, y) -> x*y));
```

Accumulator Function

Identity
Value

```
Sum of squares via reduce method: 385
Product via reduce method: 3628800
```

Stream API Example

```
//even values displayed in sorted order
```

```
System.out.printf("%nEven values displayed in sorted order: ");
```

```
IntStream.of(values).filter(value -> value % 2 == 0)
```

```
.sorted().forEach(value -> System.out.printf("%d ", value));
```

```
System.out.println();
```

```
//odd values multiplied by 10 and displayed in sorted order
```

```
System.out.printf("Odd values multiplied by 10 displayed in sorted order: ");
```

```
IntStream.of(values).filter(value -> value % 2 != 0)
```

```
.map(value -> value * 10)
```

```
.sorted()
```

```
.forEach(value -> System.out.printf("%d", value));
```

```
System.out.println();
```

```
Even values displayed in sorted order: 2 4 6 8 10
Odd values multiplied by 10 displayed in sorted order: 10 30 50 70 90
```

Stream API Example

```
//sum range of integers from 1 to 10, exclusive
System.out.printf("%nSum of integers from 1 to 9: %d%n",
    IntStream.range(1, 10).sum());
//sum range of integers from 1 to 10, inclusive
System.out.printf("Sum of integers from 1 to 10: %d%n",
    IntStream.rangeClosed(1, 10).sum());
}
} // end class IntStreamOperations
```

```
Sum of integers from 1 to 9: 45
Sum of integers from 1 to 10: 55
```

Another Stream API Example

```
public class Dish {  
    private final String name;  
    private final boolean vegetarian;  
    private final int calories;  
    private final Type type;  
  
    public Dish(String name, boolean vegetarian, int calories, Type type)  
    public String getName() {...3 lines }  
    public boolean isVegetarian() {...3 lines }  
    public int getCalories() {...3 lines }  
    public Type getType() {...3 lines }  
    @Override  
    public String toString() {...3 lines }  
    public enum Type {...}
```

Another Stream API Example

```
List<Dish> menu = Arrays.asList(  
    new Dish("beef", false, 700, Dish.Type.MEAT),  
    new Dish("chicken", false, 400, Dish.Type.MEAT),  
    new Dish("french fries", true, 530, Dish.Type.OTHER),  
    new Dish("rice", true, 350, Dish.Type.OTHER),  
    new Dish("season fruit", true, 120, Dish.Type.OTHER),  
    new Dish("pizza", true, 550, Dish.Type.OTHER),  
    new Dish("prawns", false, 300, Dish.Type.FISH),  
    new Dish("salmon", false, 450, Dish.Type.FISH) );
```

Another Stream API Example

- ▶ Consider the following code snippets

```
List<Dish> vegetarianDishes =  
new ArrayList<>();
```

```
for(Dish d: menu){  
    if(d.isVegetarian()){  
        vegetarianDishes.add(d);  
    }  
}
```

```
List<Dish> vegetarianDishes =  
menu.stream()  
.filter(d->d.isVegetarian)  
.collect(toList());
```

Using Stream API

Without using Stream API

```
100.  
vegetarianDishes [french fries, rice, season fruit, pizza]
```

Another Stream API Example

Get s Stream from menu
(the list of dishes)

Create a pipeline of
operations: first filter high-
calorie dishes.

Get the names
of the dishes

The result is []

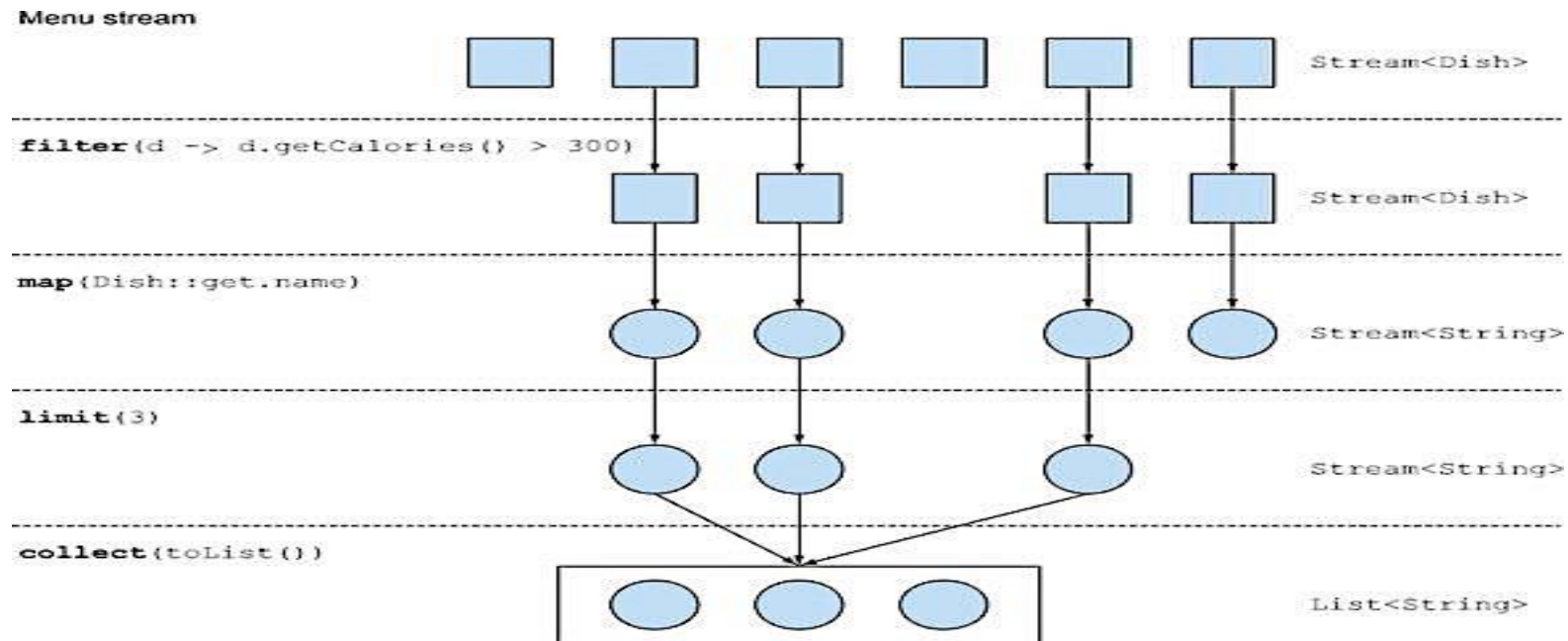
Select
only the
first
three

```
List<String> threeHighCalorieDishNames =  
    menu.stream()  
        .filter(d -> d.getCalories() > 300)  
        .map(Dish::getName)  
        .limit(3)  
        .collect(toList());  
System.out.println(threeHighCalorieDishNames);
```

Store the
result in
another
List

[beef, chicken, french fries]

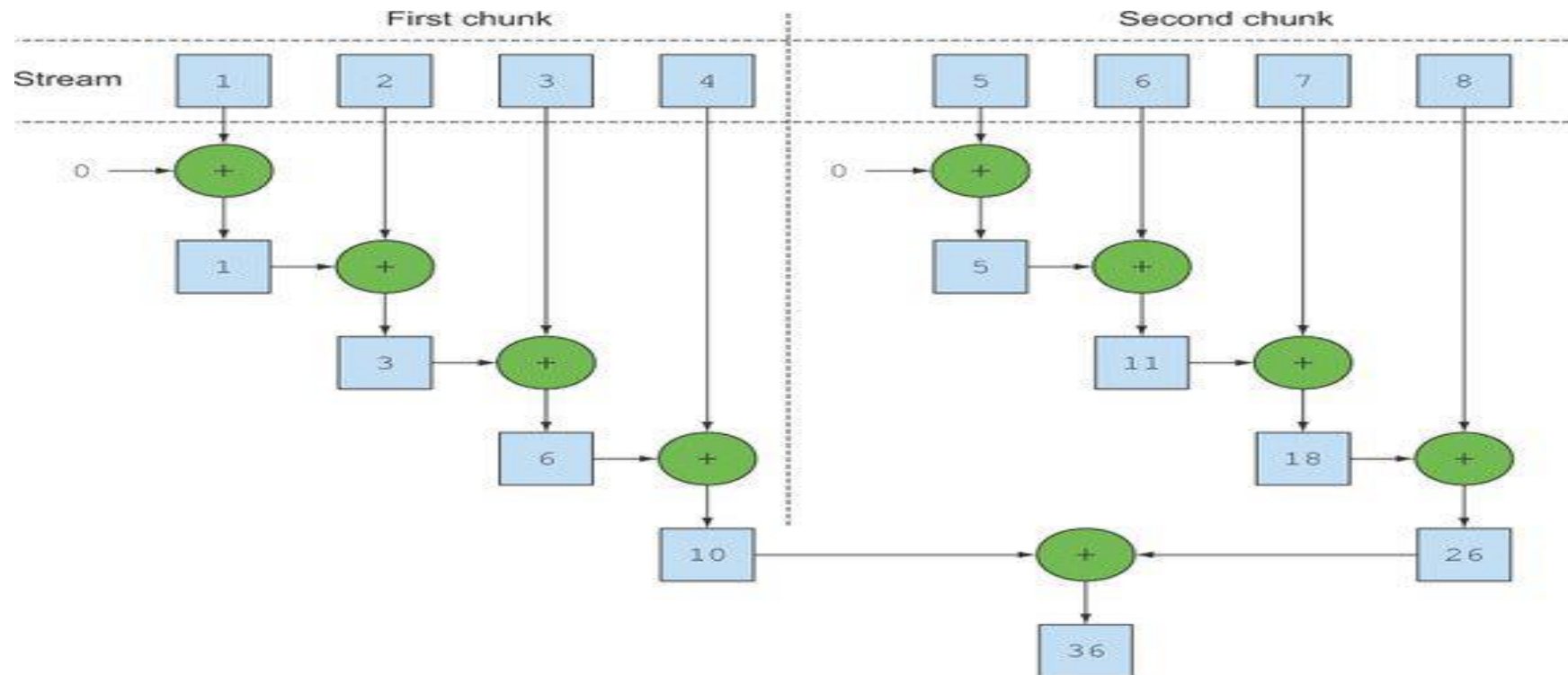
Another Stream API Example



Parallel data processing and performance

- ▶ One of the most important benefit of Stream API is the possibility of executing a pipeline of operations on collections that automatically makes use of the multiple cores on your computer.
- ▶ Processing a collection of data in parallel was extremely cumbersome.
 - ▶ First, you needed to explicitly split the data structure containing your data into subparts.
 - ▶ Second, you needed to assign each of these subparts to a different thread.
 - ▶ Third, you needed to synchronize them opportunely to avoid unwanted race conditions, wait for the completion of all threads, and finally combine the partial results.
 - ▶ Java 7 introduced a framework called fork/join to perform these operations more consistently and in a less error-prone way.

Parallel data processing and performance





Lab Exercises

Lab Exercise 1

- ▶ World Countries:
 - ▶ There are two domain classes: Country and City.
 - ▶ Each city belongs to a country defined by the attribute, countryCode.
 - ▶ Each country has a unique code and has many cities.
- ▶ The Country class has the following attributes
- ▶ The City class has the following attributes

```
public class Country {  
    private String code;  
    private String name;  
    private String continent;  
    private double surfaceArea;  
    private int population;  
    private double gnp;  
    private int capital;  
}
```

ABW	Aruba	North Am	103000	193	828	129
AFG	Afghanistan	Asia	22720000	652090	5976	1
AGO	Angola	Africa	12878000	1246700	6648	56
AIA	Anguilla	North Am	8000	96	63.2	62
ALB	Albania	Europe	3401200	28748	3205	34
AND	Andorra	Europe	78000	468	1630	55
ANT	Netherlands	North Am	217000	800	1941	33
ARE	United Arab Emirates	Asia	2441000	83600	37966	65
ARG	Argentina	South Am	37032000	2780400	340238	69
ARM	Armenia	Asia	3520000	29800	1813	126
ASM	American Samoa	Oceania	68000	199	334	54

A	B	C	D
1	Kabul	1780000	AFG
2	Qandahar	237500	AFG
3	Herat	186800	AFG
4	Mazar-e-Sharif	127800	AFG
5	Amsterdam	731200	NLD
6	Rotterdam	593321	NLD
7	Haag	440900	NLD
8	Utrecht	234323	NLD
9	Eindhoven	201843	NLD
10	Tilburg	193238	NLD
11	Groningen	172701	NLD
12	Breda	160398	NLD
13	Apeldoorn	153491	NLD
14	Nijmegen	152463	NLD
15	Enschede	149544	NLD
16	Haarlem	148772	NLD

```
public class City {  
    private int id;  
    private String name;  
    private int population;  
    private String countryCode;  
}
```

Lab Exercise 1

- ▶ Create the above two classes and then create a java application to find the following:
 - ▶ Find the highest populated city of each country
 - ▶ Find the most populated country of each continent
 - ▶ Find the highest populated capital city